

TensorFlow.js：使用卷積類神經網路辨識手寫數字

來源：<https://codelabs.developers.google.com/codelabs/tfjs-training-classification?hl=zh-tw>

步驟數：8

在本程式碼研究室中，您將訓練模型辨識手寫數字。在機器學習術語中，這稱為分類工作，因為模型能預測指定輸入內容的類別。

目錄

1. [1. 簡介](#)
2. [2. 做好準備](#)
3. [3. 載入資料](#)
4. [4. 構思工作](#)
5. [5. 定義模型架構](#)
6. [6. 訓練模型](#)
7. [7. 評估模型](#)
8. [8. 主要重點](#)

1. 簡介

在本教學課程中，我們將建構 TensorFlow.js 模型，透過卷積類神經網路辨識手寫數字。首先，我們會讓分類器「查看」數千張手寫數字圖片及其標籤，藉此訓練分類器。接著，我們會使用模型從未見過的測試資料，評估分類器的準確率。

這項工作視為分類工作，因為我們訓練模型是為了將類別 (圖片中顯示的數字) 指派給輸入圖片。我們會向模型展示許多輸入內容和正確輸出內容的範例，藉此訓練模型。這就是所謂的[監督式學習](#)。

建構目標

您將建立網頁，使用 TensorFlow.js 在瀏覽器中訓練模型。只要提供特定大小的黑白圖片，系統就會分類圖片中顯示的數字。步驟如下：

- 載入資料。
- 定義模型架構。
- 訓練模型並監控訓練期間的效能。
- 進行一些預測，評估訓練好的模型。

課程內容

- 使用 TensorFlow.js Layers API 建立卷積模型的 TensorFlow.js 語法。
- 在 TensorFlow.js 中制定分類工作
- 如何使用 tfjs-vis 程式庫監控瀏覽器內訓練。

軟硬體需求

- 新版 [Chrome](#) 或支援 ES6 模組的其他新式瀏覽器。
- 文字編輯器，可在本機電腦上執行，也可以透過 [Codepen](#) 或 [Glitch](#) 等服務在網路上執行。
- 熟悉 HTML、CSS、JavaScript 和 [Chrome 開發人員工具](#) (或您偏好的瀏覽器開發人員工具)。
- 對類神經網路有高階概念性瞭解。如需簡介或複習，建議觀看 [3blue1brown 的這部影片](#)，或是 [Ashi Krishnan 的這部 JavaScript 深度學習影片](#)。

此外，您也應熟悉[第一個訓練教學課程](#)中的教材。

步驟 2 · 約 5 分鐘

2. 做好準備

建立 HTML 網頁並加入 JavaScript



將下列程式碼複製到名為

`index.html`

〇〇〇〇

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TensorFlow.js Tutorial</title>

  <!-- Import TensorFlow.js -->
  <script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@1.0.0/dist/tf.min.js"></script>
  <!-- Import tfjs-vis -->
  <script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs-vis@1.0.2/dist/tfjs-vis.umd.min.js"></script>

  <!-- Import the data file -->
  <script src="data.js" type="module"></script>

  <!-- Import the main script file -->
  <script src="script.js" type="module"></script>

</head>

<body>
</body>
</html>
```

建立資料和程式碼的 JavaScript 檔案

1. 在與上述 HTML 檔案相同的資料夾中，建立名為 **data.js** 的檔案，然後將[這個連結](#)中的內容複製到該檔案。
2. 在與步驟一相同的資料夾中，建立名為 **script.js** 的檔案，並在其中加入下列程式碼。

```
console.log('Hello TensorFlow');
```

注意：如果您是在 CodeLab 資訊站，建議使用 glitch.com 或 codepen.io 完成本程式碼研究室。我們已設定入門專案，供您在 [Glitch](#) 上重新混音，或在 [CodePen](#) 上建立分支，載入 tensorflow.js。

注意：這些設定說明 (以及本教學課程的其餘部分) 主要著重於透過指令碼標記載入檔案。許多 JavaScript 開發人員偏好使用 npm 安裝依附元件，並使用 bundler 建構專案。如果是這種情況，您也可以從 NPM 安裝 [tensorflow.js](#) 和 [tfjs-vis](#)。

注意：視瀏覽器權限而定，您可能需要使用本機網路伺服器查看檔案，才能規避 CORS 限制。[Python SimpleServer](#) 或 [Node http server](#) 都是不錯的選擇。或者，您也可以使用 [Glitch](#) 或 [Codepen](#) 等線上程式碼平台。

立即測試

現在您已建立 HTML 和 JavaScript 檔案，請測試這些檔案。在瀏覽器中開啟 index.html 檔案，並開啟開發人員工具控制台。

如果一切正常，應該會建立兩個全域變數。`tf` 是指 TensorFlow.js 程式庫，`tfvis` 則是指 [tfjs-vis](#) 程式庫。

畫面上應會顯示「`Hello TensorFlow`」訊息，如果看到這則訊息，即可繼續下一個步驟。

3. 載入資料

在本教學課程中，您將訓練模型，學習辨識圖片中的數字，如下所示。這些圖片是來自 [MNIST](#) 資料集的 28x28 像素灰階圖片。



我們已提供程式碼，可從我們為您建立的特殊 [精靈檔案](#) (~10MB) 載入這些圖片，方便您專注於訓練部分。

歡迎研究 [data.js](#) 檔案，瞭解資料的載入方式。或者，完成本教學課程後，您也可以自行建立載入資料的方法。

提供的程式碼包含一個類別 `MnistData`，其中有兩個公開方法：

- `nextTrainBatch(batchSize)`：從訓練集傳回隨機批次的圖片和標籤。
- `nextTestBatch(batchSize)`：從測試集傳回一批圖片及其標籤

`MnistData` 類別也會執行**隨機排序**和**正規化**資料的重要步驟。

共有 65,000 張圖片，我們將使用最多 55,000 張圖片訓練模型，並保留 10,000 張圖片，以便在完成後測試模型效能。我們會在瀏覽器中完成所有操作！

如果您在 Node.js 中執行類似作業，可能會直接從檔案系統載入圖片，並使用原生圖片處理解決方案取得像素資料。

現在載入資料，並測試是否正確載入。



將下列程式碼新增至 `script.js` 檔案。

```

import {MnistData} from './data.js';

async function showExamples(data) {
  // Create a container in the visor
  const surface =
    tfvis.visor().surface({ name: 'Input Data Examples', tab: 'Input Data' });

  // Get the examples
  const examples = data.nextTestBatch(20);
  const numExamples = examples.xs.shape[0];

  // Create a canvas element to render each example
  for (let i = 0; i < numExamples; i++) {
    const imageTensor = tf.tidy(() => {
      // Reshape the image to 28x28 px
      return examples.xs
        .slice([i, 0], [1, examples.xs.shape[1]])
        .reshape([28, 28, 1]);
    });

    const canvas = document.createElement('canvas');
    canvas.width = 28;
    canvas.height = 28;
    canvas.style = 'margin: 4px;';
    await tf.browser.toPixels(imageTensor, canvas);
    surface.drawArea.appendChild(canvas);

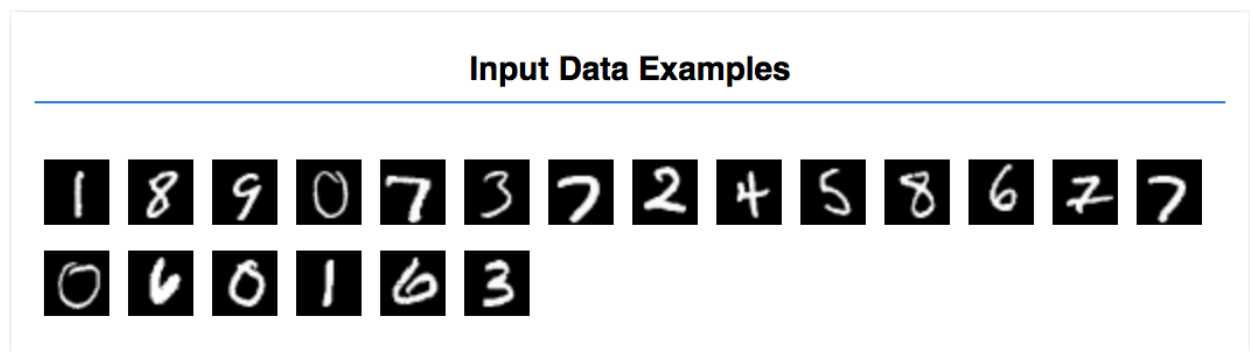
    imageTensor.dispose();
  }
}

async function run() {
  const data = new MnistData();
  await data.load();
  await showExamples(data);
}

document.addEventListener('DOMContentLoaded', run);

```

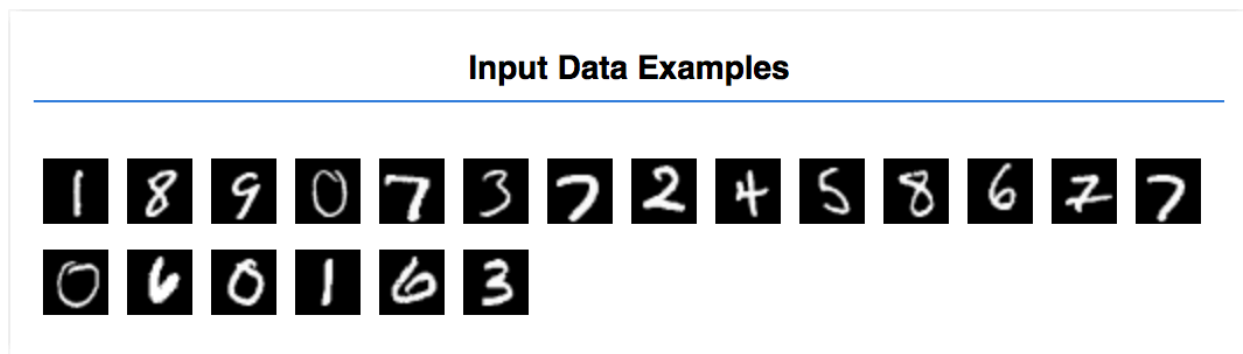
重新整理頁面，幾秒後左側應該會顯示面板，內含多張圖片。



步驟 4 · 約 3 分鐘

4. 構思工作

輸入資料如下所示。



我們的目標是訓練模型，讓模型接收一張圖片，並學習預測圖片可能屬於的 10 個類別 (數字 0 到 9) 各自的分數。

每張圖片的寬度和高度都是 28 像素，且由於是灰階圖片，因此只有 1 個顏色通道。因此每張圖片的形狀都是 `[28, 28, 1]`。

請注意，我們進行的是一對十的對應，以及每個輸入範例的形狀，因為這對下一節來說很重要。

5. 定義模型架構

在本節中，我們將編寫程式碼來描述模型架構。*模型架構是比較複雜的說法，意指「模型執行時會執行的函式」，或「模型會使用哪種演算法計算答案」。*

在機器學習中，我們會定義架構 (或演算法)，並讓訓練程序學習該演算法的參數。



在 中新增下列函式

`script.js` 檔案，用於定義模型架構

```
function getModel() {
  const model = tf.sequential();

  const IMAGE_WIDTH = 28;
  const IMAGE_HEIGHT = 28;
  const IMAGE_CHANNELS = 1;

  // In the first layer of our convolutional neural network we have
  // to specify the input shape. Then we specify some parameters for
  // the convolution operation that takes place in this layer.
  model.add(tf.layers.conv2d({
    inputShape: [IMAGE_WIDTH, IMAGE_HEIGHT, IMAGE_CHANNELS],
    kernelSize: 5,
    filters: 8,
    strides: 1,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
  }));

  // The MaxPooling layer acts as a sort of downsampling using max values
  // in a region instead of averaging.
  model.add(tf.layers.maxPooling2d({poolSize: [2, 2], strides: [2, 2]}));

  // Repeat another conv2d + maxPooling stack.
  // Note that we have more filters in the convolution.
  model.add(tf.layers.conv2d({
    kernelSize: 5,
    filters: 16,
    strides: 1,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
  }));
  model.add(tf.layers.maxPooling2d({poolSize: [2, 2], strides: [2, 2]}));

  // Now we flatten the output from the 2D filters into a 1D vector to prepare
  // it for input into our last layer. This is common practice when feeding
  // higher dimensional data to a final classification output layer.
  model.add(tf.layers.flatten());

  // Our last layer is a dense layer which has 10 output units, one for each
  // output class (i.e. 0, 1, 2, 3, 4, 5, 6, 7, 8, 9).
  const NUM_OUTPUT_CLASSES = 10;
  model.add(tf.layers.dense({
    units: NUM_OUTPUT_CLASSES,
    kernelInitializer: 'varianceScaling',
    activation: 'softmax'
  }));

  // Choose an optimizer, loss function and accuracy metric,
  // then compile and return the model
  const optimizer = tf.train.adam();
```

```
model.compile({
    optimizer: optimizer,
    loss: 'categoricalCrossentropy',
    metrics: ['accuracy'],
});

return model;
}
```

讓我們進一步瞭解這項功能。

卷積

```
model.add(tf.layers.conv2d({
    inputShape: [IMAGE_WIDTH, IMAGE_HEIGHT, IMAGE_CHANNELS],
    kernelSize: 5,
    filters: 8,
    strides: 1,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
}));
```

這裡我們使用的是序列模型。

我們使用的是 `conv2d` 層，而非稠密層。我們無法詳細說明捲積的運作方式，但以下資源會說明基礎作業：

- [以視覺化方式說明圖像核心](#)
- [Convolutional Neural Networks for Visual Recognition](#)

讓我們來細看 `conv2d` 設定物件中的每個引數：

- `inputShape`。資料的形狀，會流入模型的第一層。在本例中，MNIST 範例是 28x28 像素的黑白圖片。圖片資料的標準格式為 `[row, column, depth]`，因此我們要在這裡設定 `[28, 28, 1]` 的形狀。每個維度的像素數為 28 列和 28 欄，深度為 1，因為圖片只有 1 個顏色通道。請注意，我們不會在輸入形狀中指定批次大小。層級的設計與批量大小無關，因此在推論期間，您可以傳遞任何批量大小的張量。
- `kernelSize`。要套用至輸入資料的滑動卷積濾波器視窗大小。這裡我們將 `kernelSize` 設為 5，指定 5x5 的方形捲積視窗。
- `filters`：要套用至輸入資料的 `kernelSize` 大小篩選器視窗數量。在這裡，我們會對資料套用 8 個篩選器。
- `strides`。滑動視窗的「步距」，也就是篩選器每次在圖片上移動時會位移的像素數。這裡我們指定步幅為 1，也就是說，篩選器會以 1 像素的步幅滑過圖片。
- `activation`。這是指在完成捲積後套用至資料的活化函數。在本例中，我們套用了[線性整流函數 \(ReLU\)](#)，這是機器學習模型中非常常見的活化函數。
- `kernelInitializer`。用於隨機初始化模型權重的方法，對訓練動態非常重要。我們不會在此詳述初始化作業，但 `VarianceScaling` (在此使用) 一般是[初始化工具的合適選擇](#)。

您也可以只使用密集層建構圖片分類器，但事實證明，捲積層對於許多圖片相關工作都非常有效。

簡化資料表示法

```
model.add(tf.layers.flatten());
```

圖片是高維度資料，而卷積運算往往會增加輸入資料的大小。在將資料傳遞至最終分類層之前，我們需要將資料攤平為一個長陣列。密集層 (我們用來做為最終層) 只需 `tensor1d` 秒，因此這個步驟在許多分類工作中都很常見。

注意：平坦層中沒有權重。它只會將輸入內容展開為長陣列。

計算最終機率分配

```
const NUM_OUTPUT_CLASSES = 10;
model.add(tf.layers.dense({
  units: NUM_OUTPUT_CLASSES,
  kernelInitializer: 'varianceScaling',
  activation: 'softmax'
}));
```

我們會使用 `softmax` 啟動的稠密層，計算 10 個可能類別的機率分布。分數最高的類別就是預測的數字。

請注意，我們希望建立一對十的對應關係 (一張輸入圖片對應十個機率)。這就是輸出層有 10 個單元的原因。

Softmax 最有可能是分類工作最後一層使用的啟動函式。

選擇最佳化工具和損失函式

```
const optimizer = tf.train.adam();
model.compile({
  optimizer: optimizer,
  loss: 'categoricalCrossentropy',
  metrics: ['accuracy'],
});
```

我們會編譯模型，並指定 [最佳化工具](#)、[損失函式](#) 和要追蹤的指標。

與第一個教學課程不同，這裡我們使用 `categoricalCrossentropy` 做為損失函式。顧名思義，當模型的輸出結果為機率分布時，就會使用這個函式。`categoricalCrossentropy` 會測量模型最後一層產生的機率分布，與真實標籤提供的機率分布之間的誤差。

舉例來說，如果我們的數字確實代表 7，可能會得到下列結果

索引	0	1	2	3	4	5	6	7	8	9
True 標籤	0	0	0	0	0	0	0	1	0	0
預測	0.1	0.01	0.01	0.01	0.20	0.01	0.01	0.60	0.03	0.02

類別交叉熵會產生單一數字，指出預測向量與真實標籤向量的相似程度。

這裡使用的標籤資料表示法稱為「one-hot 編碼」，常見於分類問題。每個類別都會與每個樣本的機率相關聯。如果我們確切知道應該是什麼，就可以將該機率設為 1，其他機率則設為 0。如要進一步瞭解 One-Hot 編碼，請參閱[這個頁面](#)。

我們要監控的另一個指標是 `accuracy`，在分類問題中，這是指所有預測中正確預測所占的百分比。

步驟 6 · 約 10 分鐘

6. 訓練模型



將下列函式複製到 **script.js** 檔案。

```
async function train(model, data) {
  const metrics = ['loss', 'val_loss', 'acc', 'val_acc'];
  const container = {
    name: 'Model Training', tab: 'Model', styles: { height: '1000px' }
  };
  const fitCallbacks = tfvis.show.fitCallbacks(container, metrics);

  const BATCH_SIZE = 512;
  const TRAIN_DATA_SIZE = 5500;
  const TEST_DATA_SIZE = 1000;

  const [trainXs, trainYs] = tf.tidy(() => {
    const d = data.nextTrainBatch(TRAIN_DATA_SIZE);
    return [
      d.xs.reshape([TRAIN_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  const [testXs, testYs] = tf.tidy(() => {
    const d = data.nextTestBatch(TEST_DATA_SIZE);
    return [
      d.xs.reshape([TEST_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  return model.fit(trainXs, trainYs, {
    batchSize: BATCH_SIZE,
    validationData: [testXs, testYs],
    epochs: 10,
    shuffle: true,
    callbacks: fitCallbacks
  });
}
```



接著，將下列程式碼加到

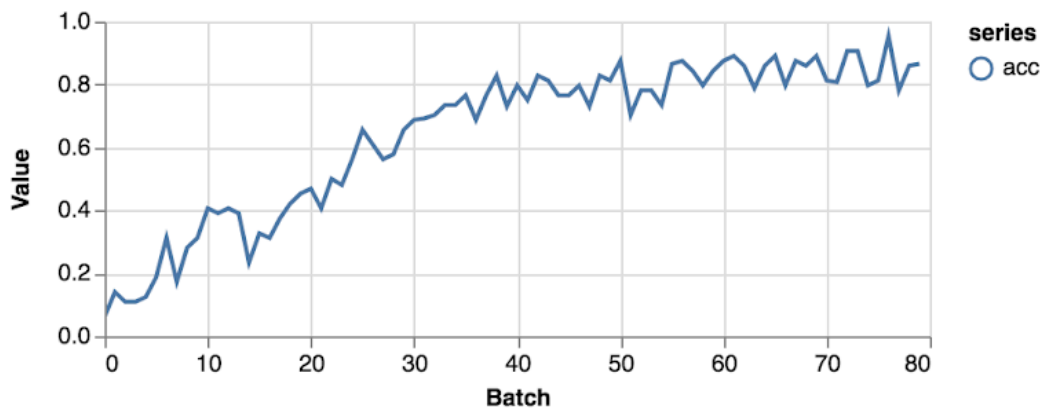
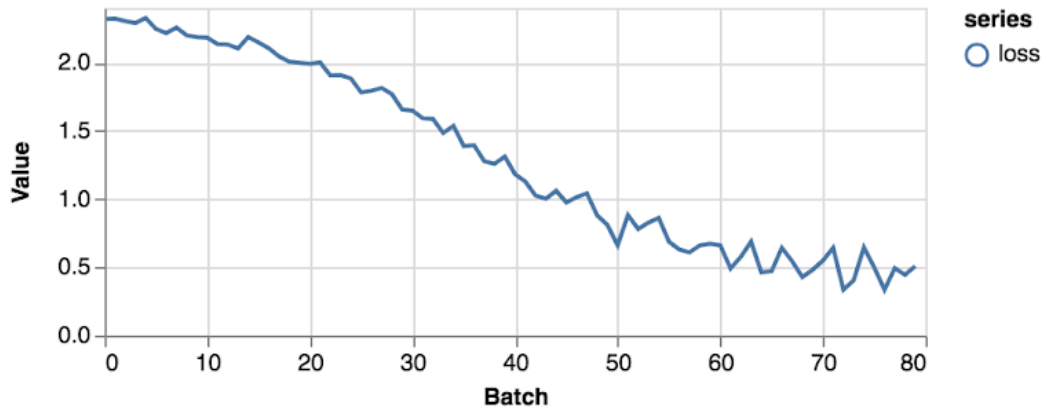
run 函式。

```
const model = getModel();  
tfvis.show.modelSummary({name: 'Model Architecture', tab: 'Model'}, model);  
  
await train(model, data);
```

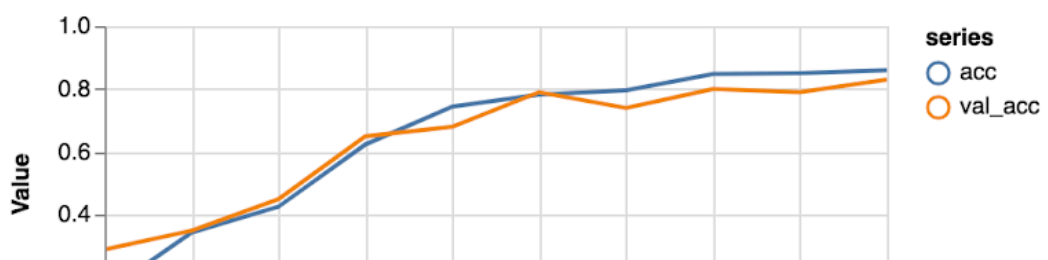
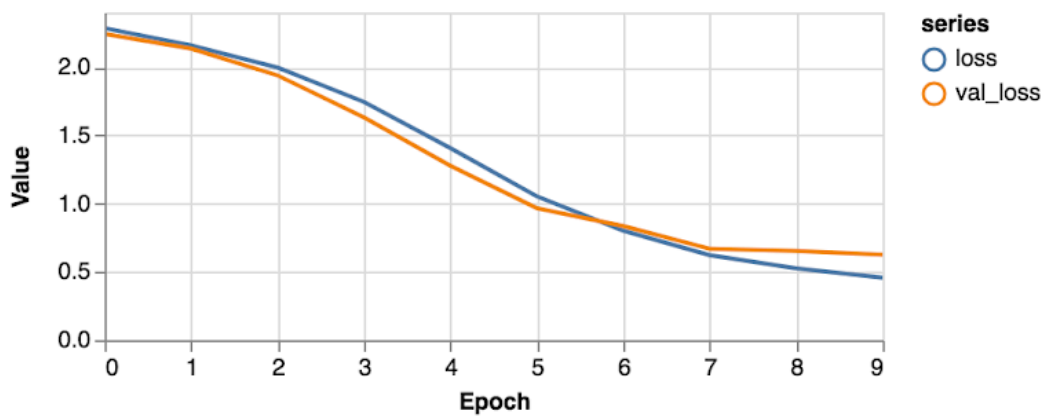
重新整理頁面，幾秒後您應該會看到一些圖表，顯示訓練進度。

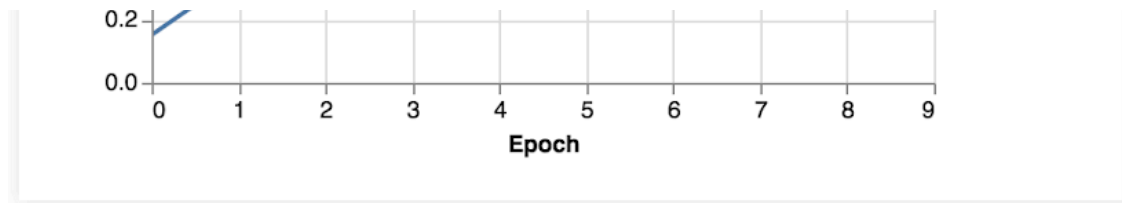
Model Training

onBatchEnd



onEpochEnd





讓我們來深入瞭解。

監控指標

```
const metrics = ['loss', 'val_loss', 'acc', 'val_acc'];
```

我們將在此決定要監控哪些指標。我們會監控訓練集的損失和準確率，以及驗證集的損失和準確率 (分別為 `val_loss` 和 `val_acc`)。我們會在下文進一步說明驗證集。

注意：使用 Layers API 時，系統會針對每個批次和週期計算損失和準確率。

以張量形式準備資料

```
const BATCH_SIZE = 512;
const TRAIN_DATA_SIZE = 5500;
const TEST_DATA_SIZE = 1000;

const [trainXs, trainYs] = tf.tidy(() => {
  const d = data.nextTrainBatch(TRAIN_DATA_SIZE);
  return [
    d.xs.reshape([TRAIN_DATA_SIZE, 28, 28, 1]),
    d.labels
  ];
});

const [testXs, testYs] = tf.tidy(() => {
  const d = data.nextTestBatch(TEST_DATA_SIZE);
  return [
    d.xs.reshape([TEST_DATA_SIZE, 28, 28, 1]),
    d.labels
  ];
});
```

這裡我們建立兩個資料集：一個是訓練集，用來訓練模型；另一個是驗證集，用來在每個訓練週期結束時測試模型，但驗證集中的資料在訓練期間不會顯示給模型。

我們提供的資料類別可輕鬆從圖片資料取得張量。不過，我們仍會將張量重塑為模型預期的形狀 `[num_examples, image_width, image_height, channels]`，然後再將這些張量提供給模型。每個資料集都有輸入內容 (X) 和標籤 (Y)。

注意：`trainDataSize` 設為 5500，`testDataSize` 設為 1000，方便您更快進行實驗。執行本教學課程後，請隨意將這些值分別增加至 55000 和 10000。訓練時間會稍長，但仍可在許多電腦的瀏覽器中運作。

```
return model.fit(trainXs, trainYs, {  
    batchSize: BATCH_SIZE,  
    validationData: [testXs, testYs],  
    epochs: 10,  
    shuffle: true,  
    callbacks: fitCallbacks  
});
```

我們呼叫 `model.fit` 來啟動訓練迴圈。我們也會傳遞 `validationData` 屬性，指出模型應在每個週期後用來測試自己的資料 (但不會用於訓練)。

如果模型在訓練資料上表現良好，但在驗證資料上表現不佳，表示模型可能過度配適訓練資料，無法妥善一般化先前未見的輸入內容。

7. 評估模型

驗證準確率可有效估算模型處理先前未見資料的成效 (只要該資料與驗證集在某方面相似)。不過，我們可能需要更詳細的各類別成效細目。

tfjs-vis 中有幾種方法可協助您完成這項操作。



將下列程式碼新增至 **script.js** 檔案底部

```
const classNames = ['Zero', 'One', 'Two', 'Three', 'Four', 'Five', 'Six', 'Seven', 'Eight', 'Nine'];

function doPrediction(model, data, testDataSize = 500) {
  const IMAGE_WIDTH = 28;
  const IMAGE_HEIGHT = 28;
  const testData = data.nextTestBatch(testDataSize);
  const testxs = testData.xs.reshape([testDataSize, IMAGE_WIDTH, IMAGE_HEIGHT, 1]);
  const labels = testData.labels.argmax(-1);
  const preds = model.predict(testxs).argmax(-1);

  testxs.dispose();
  return [preds, labels];
}

async function showAccuracy(model, data) {
  const [preds, labels] = doPrediction(model, data);
  const classAccuracy = await tfvis.metrics.perClassAccuracy(labels, preds);
  const container = {name: 'Accuracy', tab: 'Evaluation'};
  tfvis.show.perClassAccuracy(container, classAccuracy, classNames);

  labels.dispose();
}

async function showConfusion(model, data) {
  const [preds, labels] = doPrediction(model, data);
  const confusionMatrix = await tfvis.metrics.confusionMatrix(labels, preds);
  const container = {name: 'Confusion Matrix', tab: 'Evaluation'};
  tfvis.render.confusionMatrix(container, {values: confusionMatrix, tickLabels: classNames});

  labels.dispose();
}
```

這段程式碼的作用是什麼？

- 進行預測。
- 計算準確度指標。
- 顯示指標

接著我們將進一步說明各項步驟。

預測

```
function doPrediction(model, data, testDataSize = 500) {
  const IMAGE_WIDTH = 28;
  const IMAGE_HEIGHT = 28;
  const testData = data.nextTestBatch(testDataSize);
  const testxs = testData.xs.reshape([testDataSize, IMAGE_WIDTH, IMAGE_HEIGHT, 1]);
  const labels = testData.labels.argmax(-1);
  const preds = model.predict(testxs).argMax(-1);

  testxs.dispose();
  return [preds, labels];
}
```

首先，我們需要進行一些預測。我們將在此擷取 500 張圖片，並預測圖片中的數字 (您稍後可以增加這個數字，以便測試更多圖片)。

特別是 `argmax` 函式，可提供機率最高的類別索引。請注意，模型會輸出每個類別的機率。我們會找出機率最高的結果，並將其指派為預測結果。

您可能也會發現，我們可以一次對所有 500 個範例進行預測。這就是 TensorFlow.js 提供的向量化功能。

注意：我們不會在此使用任何機率門檻。即使值相對較低，我們也會採用最高值。這項專案的有趣擴充功能是設定一些必要的最低機率，並在沒有任何類別符合這個分類門檻時，指出「找不到數字」。

doPredictions 說明模型訓練完成後，如何一般性地進行預測。但新資料不會有任何現有標籤

顯示各類別的準確率

```
async function showAccuracy() {
  const [preds, labels] = doPrediction();
  const classAccuracy = await tfvis.metrics.perClassAccuracy(labels, preds);
  const container = { name: 'Accuracy', tab: 'Evaluation' };
  tfvis.show.perClassAccuracy(container, classAccuracy, classNames);

  labels.dispose();
}
```

有了預測和標籤，我們就能計算每個類別的準確率。

顯示混淆矩陣

```
async function showConfusion() {
  const [preds, labels] = doPrediction();
  const confusionMatrix = await tfvis.metrics.confusionMatrix(labels, preds);
  const container = { name: 'Confusion Matrix', tab: 'Evaluation' };
  tfvis.render.confusionMatrix(container, {values: confusionMatrix, tickLabels: classNames});

  labels.dispose();
}
```

混淆矩陣與每個類別的準確率類似，但會進一步細分，顯示分類錯誤的模式。這有助於瞭解模型是否會混淆任何特定類別組合。

顯示評估結果



在 `run` 函式的底部新增下列程式碼，顯示評估結果。

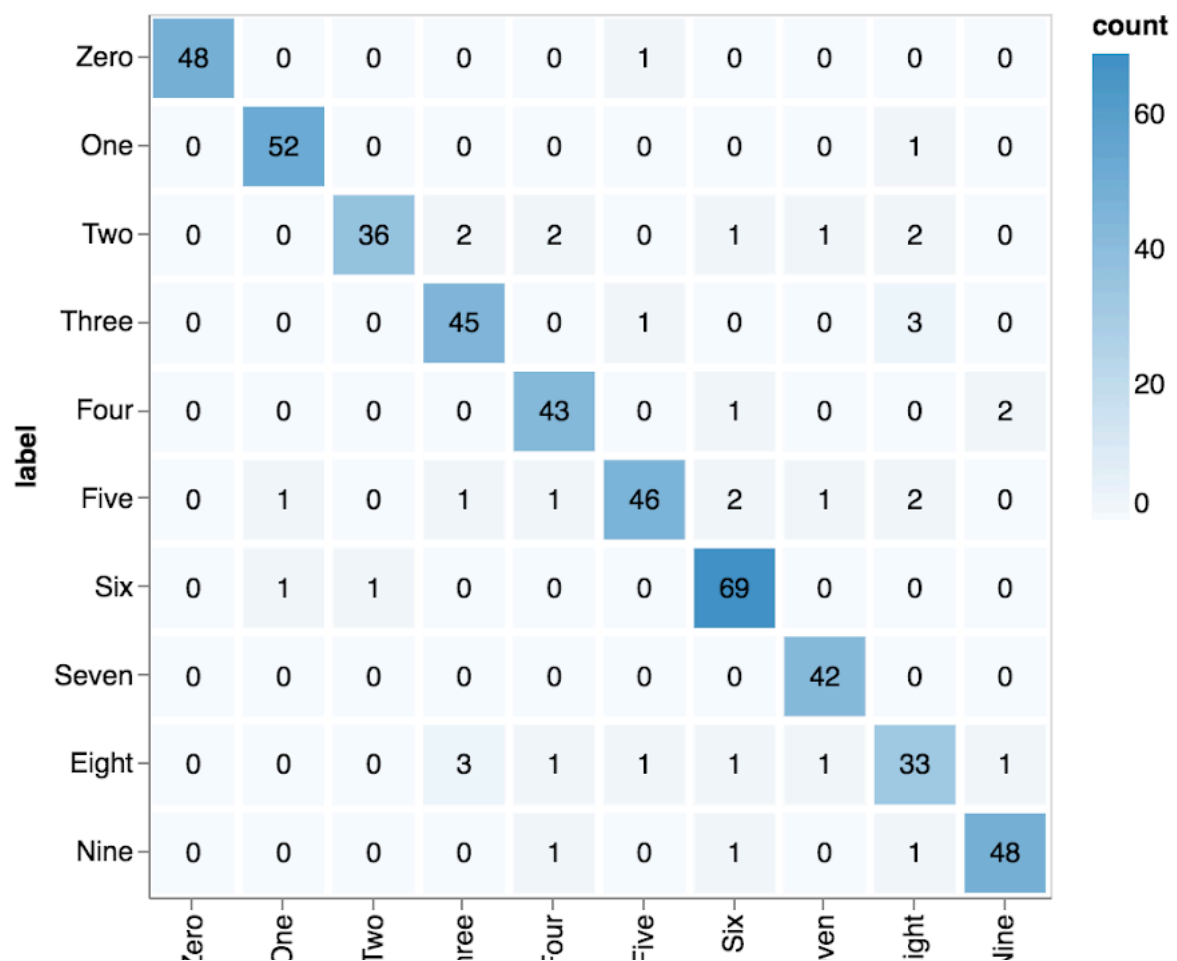
```
await showAccuracy(model, data);
await showConfusion(model, data);
```

畫面會顯示類似以下的內容。

Accuracy

Class	Accuracy	# Samples
Zero	0.9672	61
One	0.9851	67
Two	0.9048	42
Three	0.9322	59
Four	0.85	40
Five	0.881	42
Six	0.88	50
Seven	0.9787	47
Eight	0.9184	49
Nine	0.907	43

Confusion Matrix



prediction

8. 主要重點

預測輸入資料的類別稱為分類工作。

分類工作需要適當的資料表示法，才能顯示標籤

- 常見的標籤表示法包括類別的 one-hot 編碼

準備資料：

- 建議保留部分資料，不要在訓練期間提供給模型，以便評估模型。這就是所謂的驗證集。

建構及執行模型：

- 卷積模型在圖像工作方面表現優異。
- 分類問題通常會使用類別交叉熵做為損失函式。
- 監控訓練，查看損失是否下降，準確率是否上升。

評估模型

- 訓練模型後，請決定評估方式，瞭解模型在解決初始問題方面的成效。
 - 與整體準確率相比，每個類別的準確率和混淆矩陣可提供更精細的模型成效細目。
-